UNIT 15 75 min

The Standard Library Toolkit

Counter, defaultdict, dates, paths, regular expressions, and itertools: tools you will reach for in every data project.

By the end of this unit you will be able to

- Count items with `collections.Counter` and group with `defaultdict`
- Do date arithmetic with `datetime` and format dates as text
- Build file paths with `pathlib`
- Search and extract text with regular expressions (`re`)
- Combine items with `itertools` product and combination helpers

Why the standard library matters

Before installing anything, check the standard library. Most small tasks have a built-in answer, and the built-in version is documented, tested, and available on every machine. Unit 10 covered how to import and read docs; this unit covers the tools you will use most.

collections.Counter and defaultdict

`Counter` counts hashable items. It replaces the counting loop from Unit 9. `defaultdict(list)` gives every new key an empty list automatically, so you can append without checking first.

collections_demo.py

```
from collections import Counter, defaultdict

words = "the cat and the hat and the bat".split()
counts = Counter(words)
print(counts.most_common(2))

by_length = defaultdict(list)
for w in words:
    by_length[len(w)].append(w)
print(dict(by_length))
```

> **Note**
> `most_common(n)` returns the n most frequent items with their counts. When two items tie, the one that appeared first comes first.

datetime: dates and times

`date` holds a calendar day, `datetime` holds a day and a time, and `timedelta` represents a difference. Subtracting two dates gives a `timedelta`. `strftime` formats a date as text, and `strptime` parses text into a date.

dates.py

```
from datetime import date, timedelta, datetime

start = date(2026, 8, 26)
due = start + timedelta(days=21)
print(due)
print((due - start).days)
print(due.strftime("%B %d, %Y"))

parsed = datetime.strptime("2026-09-01 14:30", "%Y-%m-%d %H:%M")
print(parsed.hour)
```

Code	Meaning	Example output
`%Y`	four-digit year	2026
`%m` / `%d`	month / day, zero-padded	09 / 01
`%B`	full month name	September
`%H:%M`	hour (24-hour) and minute	14:30

> **Watch out**
> Naive `datetime` objects have no time zone. Mixing them with timezone-aware ones raises an error. Store times in one zone (often UTC) and convert only for display.

pathlib: paths that work everywhere

`Path` objects join folders with `/`, which works on Windows, macOS, and Linux. They also know how to check whether a file exists and how to read it.

pathlib_demo.py

```
from pathlib import Path

folder = Path("data")
grades = folder / "grades.csv"
print(grades)
print(grades.suffix)
print(grades.stem)
print(grades.parent)
```

Paths are only descriptions. Use `grades.exists()` to check, and `grades.read_text()` or `grades.write_text()` to work with the contents.

re: regular expressions

A regular expression describes a pattern of text. The `re` module finds, checks, and replaces text that matches. Write patterns as raw strings (`r"..."`) so backslashes reach the regex engine unchanged.

Pattern	Matches
`\d`	one digit (0–9)
`\w`	a letter, digit, or underscore
`\s`	whitespace
`.`	any character except newline
`+` / `*`	one or more / zero or more of the item before it
`[a-z]`	any lowercase letter
`^` / `$`	start / end of the text

re_demo.py

```
import re

log = "Order 1042 shipped to zip 30041 on 2026-08-26"

print(re.findall(r"\d+", log))
print(re.search(r"\d{4}-\d{2}-\d{2}", log).group())
print(re.sub(r"\d", "#", "room 204"))

m = re.match(r"(\w+)@(\w+)\.com", "ada@school.com")
print(m.group(1), m.group(2))
```

> **Tip**
> Start with a simple pattern and test it on a few examples. Regular expressions are powerful, and they are also easy to get wrong. Keep them short, and comment what they match.

itertools: combining items

`itertools` has fast tools for combinations. `product` gives every pair, `combinations` gives each unordered group once, and `permutations` gives each ordering.

itertools_demo.py

```
from itertools import product, combinations

sizes = ["S", "M"]
colors = ["red", "blue"]
print(list(product(sizes, colors)))

team = ["Ana", "Ben", "Cy"]
print(list(combinations(team, 2)))
```

Predict the output THINK FIRST

Watch for ties, zero-padding, and what `re` returns when nothing matches.

```
from collections import Counter
print(Counter("aab").most_common(1))
```

Answer: `[('a', 2)]`

```
from datetime import date, timedelta
print((date(2026, 1, 1) + timedelta(days=31)).isoformat())
```

Answer: `2026-02-01`

```
from pathlib import Path
print(Path("a/b/c.txt").suffix)
```

Answer: `.txt`

```
import re
print(re.findall(r"\d+", "a1b22c333"))
```

Answer: `['1', '22', '333']`

```
import re
print(re.search(r"z", "abc"))
```

Answer: `None`
`search` returns None when nothing matches.

```
from itertools import combinations
print(len(list(combinations([1, 2, 3, 4], 2))))
```

Answer: `6`

Practice

▼ **Exercise 1 — Most common letter**

Using `Counter`, print the most common letter in `"mississippi"` and how many times it appears.

▼ **Exercise 2 — Extract emails**

Given `text = "Contact ada@school.org or ben@example.com today."`, use `re.findall` to print a list of the two addresses. A simple pattern is `r"[\w.]+@[\w.]+"`.

Discussion

1. Which standard-library tool from this unit would have saved you the most time in a past project?
2. Regular expressions are hard to read. When is that cost worth paying, and when should you use plain string methods instead?
3. Why does pathlib use `/` to join paths? What would a hand-written path join need to handle?

Key terms

Counter

A dictionary subclass from `collections` that counts how often each item appears.

defaultdict

A dictionary that creates a default value (such as an empty list) for any missing key.

timedelta

A duration: the difference between two dates or times.

strftime / strptime

Methods that format a date as text (`strftime`) or parse text into a date (`strptime`).

Path

An object from `pathlib` that represents a file or folder path and offers methods to read, write, and inspect it.

Regular expression

A pattern that describes a set of strings, used to search, match, and replace text.

Raw string

A string literal prefixed with `r` that treats backslashes literally, commonly used for regex patterns.

itertools

A standard-library module of fast, memory-efficient tools for looping and combining items.

See all terms in the glossary →